ASC

7-IV-2024

sistem de calcul: CPU, memorie, magistrală

PMS = processor memory switches

SN (structural):

Pc = procesor central
M = memorie
S = comutator
L = linie legătură
K = unitate comandă
Pi/E = procesor intrare/ieşire sau interfaţă
D = operator de date
T = terminal

SN = sistem numeric

interfeţe paralele
⇒ interferenţe
serial → viteză mai mare

primitive PMS: – caract. de atribut (cu o anumită valoare)

SN ($a_1 : v_1$; $a_2 : v_2$, ... $a_n : v_n$)

ex: frecvenţa: 5 GHz, set instr.

frecv. nu mai creşte
⇒ nr. core-uri ↗

RISC/CISC (tip arhitectură)
≠ dim. instr. fixă/variabilă

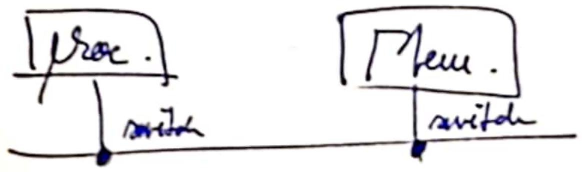
au anumite instr. de bază comune, fără de care nu se poate construi un procesor

set minimal cu care poate fi tradus orice program

optimizare în procesor: pipeline (exec în paralel a mai multor instr.)

paralelism: redundanţă hardware

– mem: frecvenţă, capacitate, latenţă de acces
(→ tip SRAM/DRAM)



mag.
- control (citire/scriere)
- date
- adrese

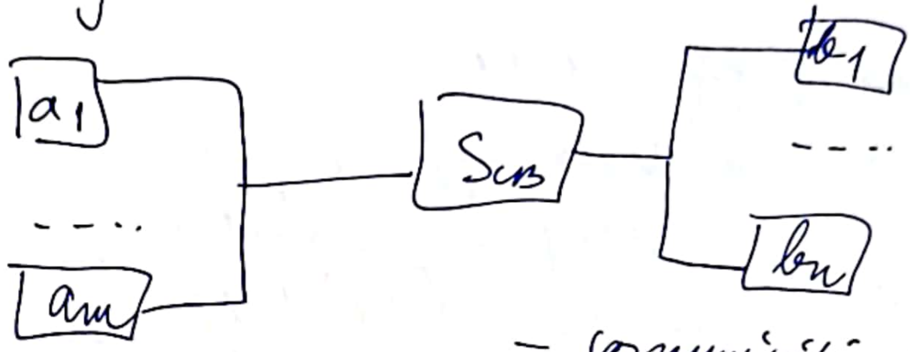
RAM: nu se parcurge secvenţial
se ştie exact unde sunt datele în memorie

ASC

14.III.2024

comutatoare ierarhice

- poarta : conectare ⎡ resursă – magistrală
  ⎣ magistrală – magistrală
- cu legătură multiplă (cross-bar switch)



– comunicații paralele

orice a poate comunica cu orice b

transferuri multiple și simultane

$S\{CB, ma, nb, c = \min(m,n), m \cdot n\, Sp\}$

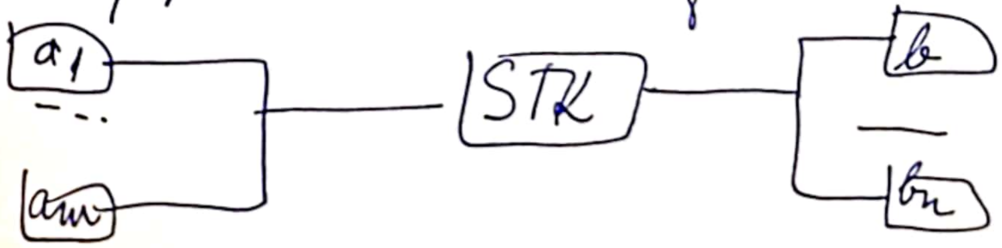
- trunchi k : conectivitate de S leg. multiplă și cost redus al celorlalte tipuri

$S\{TK, ma, nb, c = k, k(m+n)\, Sp\}$

orice a comunică cu orice b

nu întotdeauna vor fi min(a,b) com. simultane

în fct. de m, n și trafic se det. un k optim



comutatoare neierarhice:

- conex. între | elem. de același tip
  - → pe ambele sensuri, nesimultan
  - duplex (bus înlănțuit)
  - tip central (leg. multiplă)
  - trunchi k (m.k switch-uri)
- asigură infrastructura de comunicație

- legătura L: fizică între componente
  nu prelucrează informația, ci doar asigură transferul spațial al datelor
  magistrală / interfață (serială / paralelă)

(poate fi chiar un mic procesor)

unitatea de comandă: funcție de comandă a resurselor asociate unităților funcționale
- control în diverse subsegmente ale sistemului

DMA = comutarea directă a perifericelor cu memoria (fără trecere prin procesor).
- datele sunt transferate bloc cu bloc
- notificare procesor la încheierea transferului (întrerupere)

= „procesor intrare-ieșire"
- viteză
- nu se ține procesorul ocupat

- asemănător cu canalul de intrare-ieșire
  - → resurse private, locale, globale

SBC = single-board computer (procesor + mem + interfețe I/O)
↳ un nod dintr-o rețea de calculatoare (ele comunică)

ASC

21 · III · 2024

SBC = single-board computer : procesor + memorie + interfețe i/o

ac. mem. accesată de mai multe procesoare

K bus = unit. de comandă a magistralei (prioritate mai mare pt. procesor ; o dată ales masterul, ceilalți așteaptă)

mem. nu poate fi master

mecanism arbitrare — rețea serială : com. / sg. perechi/entitate o dată

paralelism pe nivel de

- bit (floating point op.)
- instrucțiune (pipeline)
- memorie (suprapunere op. cu mem și cele de calcul)
- sist. de operare (thread-uri, CPU-uri multiple)
- distribuite (mașini conectate împreună)

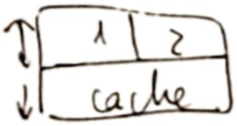
CPU - north bridge - DRAM

- roofline model (arată ca un acoperiș grafic) — câte op. floating point pot face pe 1 octet (nr. aduse din mem. la procesor) (câți flops)

GPU - highly parallel ; menține curba legii lui Moore

nr. tranzistor / core < nr. tranzistor / chip → loc cache ↗ proces

multicore :



pipeline → hazard de control → branch predictor

data → out-of-order scheduler

[illegible]

core GPU ≠ core CPU

↳ anumite programe

↳ orice program

problene

- +consum power (încălzire)
- mem. (reproiectare ierarhie)
- ILP

⇓

exploatare paralelism la nivelul aplicației

(instruction-lever parallelism

(x86) SIMD

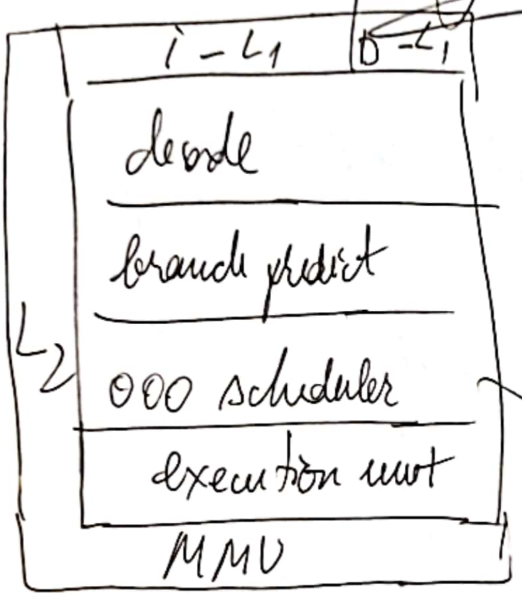
↳ pipeline

hyperthreading

(superscalar / op. tipuri diferite

hazarde

optimizări



SIMD → 1 unitate de comandă

ac. instr, mai multe date (unit. de exec.)

~~compilator~~ → compilator ce elimină hazarde de date

! registre în plus >> L3 din cache

(viteză mai mare)

* fine-grained interleaved threading

- cache size ↓
- X branch predictor
- X OOO scheduler

avantaje

legiră mai simplă, mai multe calcule (eficiență op. față de date)

dezavantaj

- paralelism ↗↗
- thread scheduler
- presiune pe registre

ASC 26.III.2024

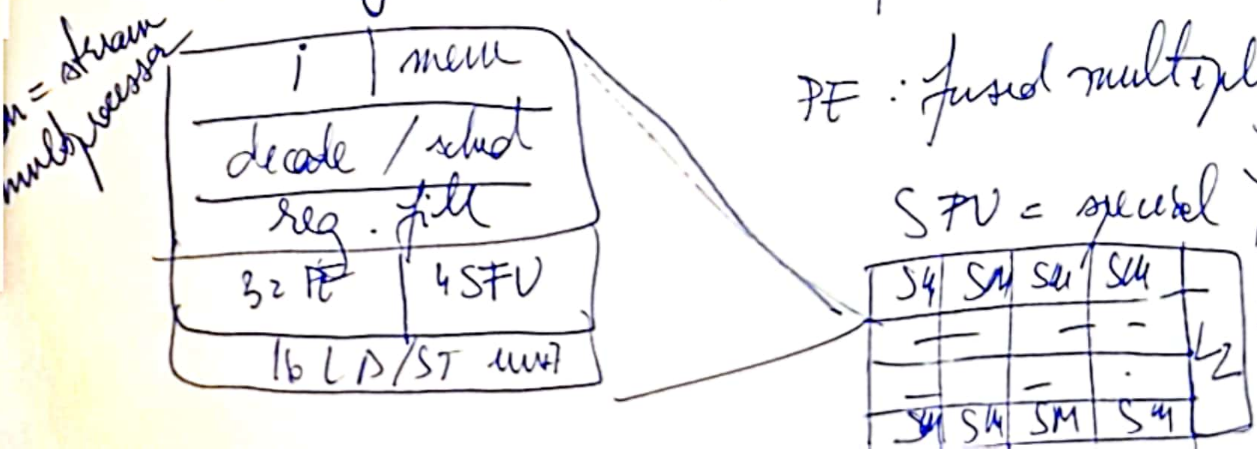
lansare de programe din CPU în GPU

CPU: cache ↗

pentru nvidia:
- fără OOO scheduler → ne bazăm pe compilatoare
  - → salvare spațiu pe chip — că nu avem memorie de ceva dedica
  - (eliminăm hazarde de date la compilare)
- fără branch prediction (căi de urmare cod de control pe GPU) ← e mutat pe CPU, de aia s-a renunțat la el
- core simplificat: cache ↘
- mai multe thread-uri ⊛ decât core-uri
- file register (nu are overhead la context switch-uri) (să știe să sare la grupul de regiștri necesar

⊛ unele thread-uri execută, altele aduc date în memorie (nu se pierde timp stând degeaba) (+ să nu existe suprapuneri → paralelism cât mai mare)

SIMD = single-instruction, multiple data


SM = stream multiprocessor
i | mem
decode / sched
reg. file
32 PE | 4 SFU
16 LD/ST unit
SM SM SM SM
L2
SM SM SM SM

PE: fused multiply add (FMA)

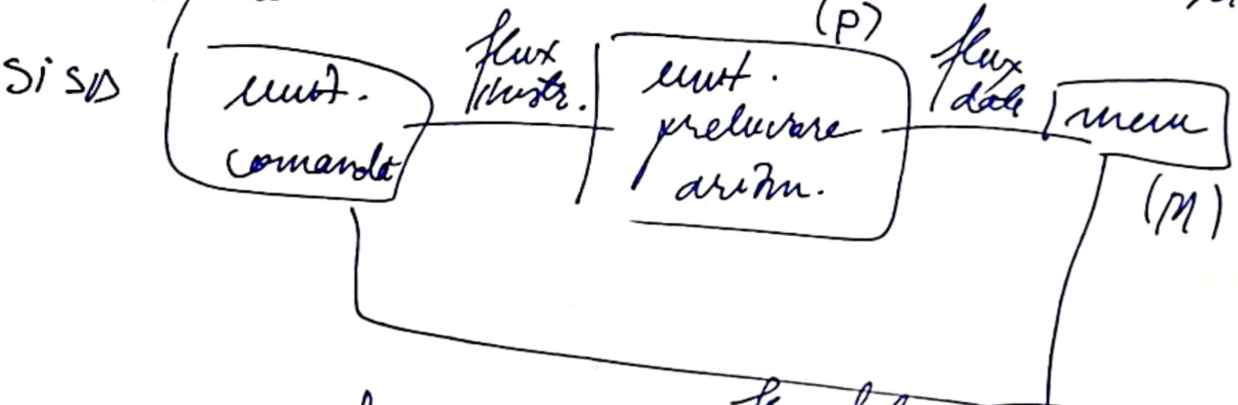
SFU = special function unit

ASC 4·IV·2024

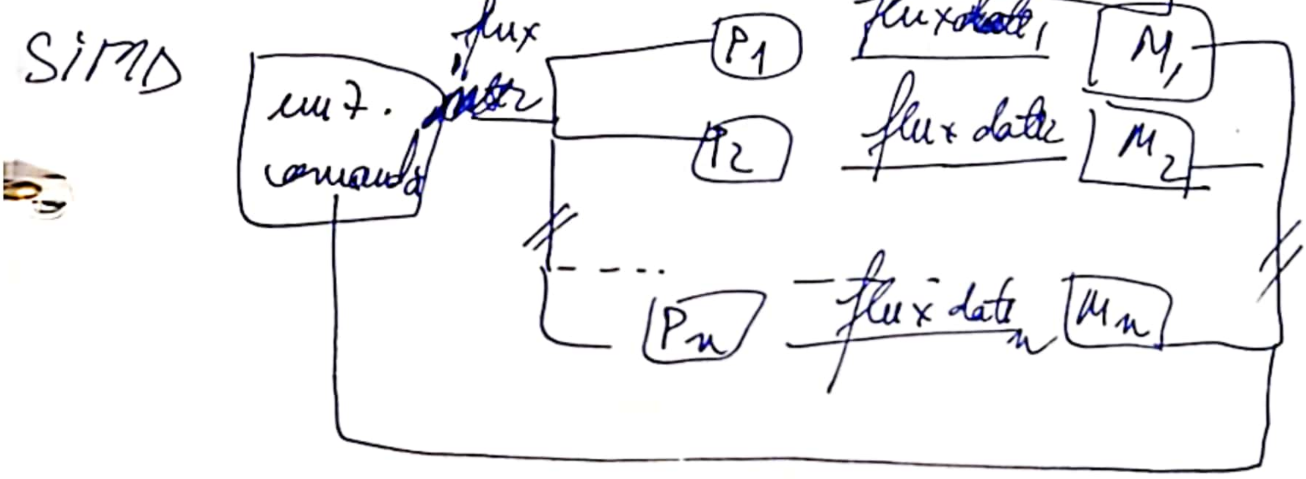
- tipuri paralelism:
  - nivel de
    - bit (floating-point op.)
    - instrucțiune (>1 instr./ciclu)
    - memorie (suprapunere între op. cu memoria și de calcul)
  - al SO → thread-uri, procese pe CPU-uri multiple pe aceeași mașină
  - distribuit → mai multe mașini conectate împreună
- flux
  - date
  - instrucțiuni

- clasificare sisteme de calcul: SISD, SIMD, MISD, MIMD

SISD


unit. comandă
flux instr.
(P)
unit. prelucrare aritm.
flux date
mem
(M)

SIMD


unit. comandă
flux instr
P1
P2
Pn
flux date1
flux date
flux date n
M1
M2
Mn

MIMD: comunicare P-P sau P-M
toate proc. participă la ac. program
mod programare: shared memory
distributed memory (transfer de mesaje)

GPU = SIMT + thread

- nu e nevoie de sincronizare la nivel de instrucțiune (la core-uri)
- MISD: aplicarea alg. diferiți pe aceleași date → meteo / evidența populației
  (aplicabilitate restrânsă)
- SIMD: mașini vectoriale

ASC 9.IV.2024

structura SIMD

- structuri adecvate prelucrării vectoriale
- vcmd: citește, interpretează și exec. instr. de control
  ↳ + ramificație
  reg. generale rapide
- P: sincronizate la nivel de instrucțiune; fiec. procesor → alt set de date (reg. locale)
- M: n module; conține instr de comandă și prelucrare + date

instr. asamblare: load A / load A [i]

sto A — memorare

add A [i] — adunare la acumulatori

mul A [i]

difuzie scalară ← bcast i — broadcast la toate celulele

index [i] — linia la care am ajuns

inițializare indice în for:

Index [i] = ...

enxc i, const

ldnx i, adr (mem [adr])

icnx i, const index += const

cpnx i, j, adr (if index [i] < index [j] then jump adr)

If pe SIMD [mascare a activităților procesoarelor
nu în aceeași etapă în else

mascare P → activare P → mascare P → activare P

- repartizare în memorii + acces paralel
- citire vectori la o singură instrucțiune

Înmulțire de matrice

```
for (i=1; i<=n; i++)
    $c_{ik} = 0 \quad (k=\overline{1,...n})$
    for (j=1; i<=n; j++)
        $c_{ik} = c_{ik} +$
            $+ a_{ij} * b_{jk} \quad (k=\overline{1,...n})$
```

scăpăm de asta: în paralel pe fiec. proc.

```
for(i=0; i<n; i++)
~~for(j=0; j<n; j++)~~
    for(k=0; k<n; k++)
        c[i][j] +=
        a[i][k] *
        * b[k][j]
```

fiec. proc: alege linia (înainte) | și citește b-ul în paralel

același pt. toate ⇒ broadcast la celelalte proc.

la fiec. pas am nevoie de alt a_{ik}

- la reprez. directă pe linii: load A[i]: citire coloana i printr-un singur acces
→ paralel pe linii; secv. pe coloane

- reprez. directă pe linii/coloane sunt incompatibile la același moment de timp

- reprezentare deplasată ciclică: acces paralel la liniile și coloanele matricii
= shuffling-ul în rețeaua de comunicație (nu se citește mereu de la ac. zonă de mem.)
↓ permutare circulară pe linii (la stânga)

salturi condiționale prin mascare

—

11. IV. 2024

SIMD: sincronizări implicite după fiecare instrucțiune (toate durează la fel de mult)

- circuit crossbar: nu scalează + e scump
- rețele de comunicație: ∀ in conectat la ∀ out
  - 1 intrare poate fi conectată simultan la mai multe ieșiri (bcast)
  - 1 ieșire poate fi conectată la 1 singură intrare

concurența $\overset{=\min}{\max}(m, n)$

- GCN = generalised connection network
  - întârziere propagare Δt)
  - complexitate (nC): nr. ~~tot~~ de comutatoare elementare utilizate în implementare

=> subrețele de comutare organizate ierarhic, pe mai multe niveluri: $k \simeq \log_2 n$ comutatoare => $nC \downarrow$; $\Delta t \nearrow$)

tip crossbar $a \times b$ cu a și $b \ll m$

între nivele => permutări de interconectare

- crossbar cu n·m comutatoare ($\Delta t - 1$)
  = respect. 2 intrări / 1 ieșire
  = com. directă
  = $nC = n \cdot m$
  = nu se pretează la VLSI
  = nu se poate bcast
  = inflexibil la dezv. ulterioară

1 switch închis pe linie și 1 închis pe coloană

- structura Bezier: rețea n×n cu 2 module $n/2 \times n/2$ și 4n module tip 2×2
  complex: $4n \log n - 2n$

desen?

- rețea Delta: n nivele; a^n intrări b^n ieșiri
  niv. 0 => a^{n-1}
  niv i: $a^{n-i} b^{i-1}$ module tip a×b
  crossbar-uri a×b și perm. de tip intercalare perfectă
  => total: $a^n - b^n$

ASC 18.IV.2024

Arhitectura Cell ← SRAM

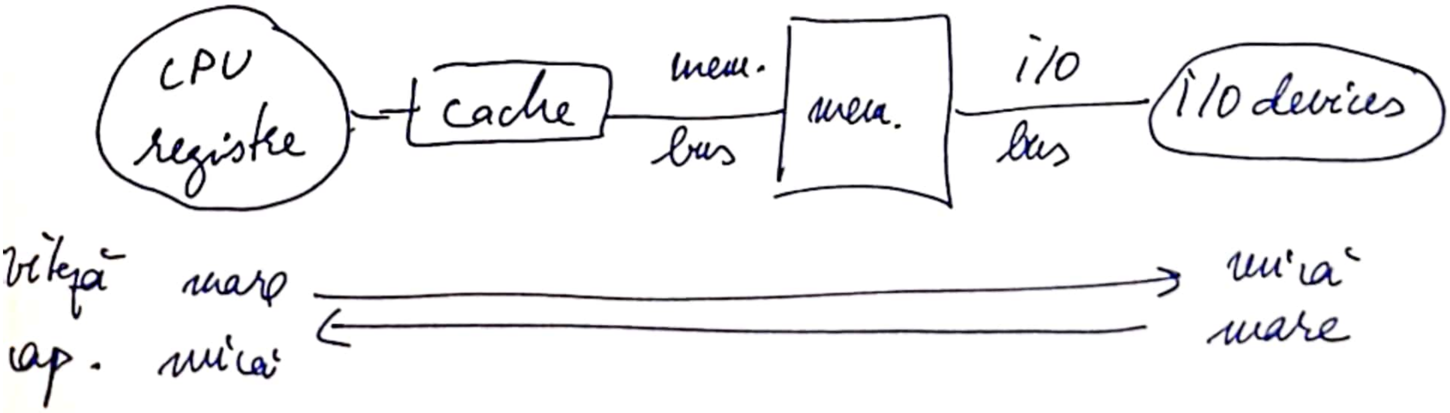
- local store memory : trebuie să încapă programul acolo
  (transfer pagină cu pagină
- arh. eterogenă de core-uri
- branch prediction
- unități de execuție diferite (→ hyperthreading)

PPE = power processor element

2 core-uri general purpose

Cell	GPU + CPU
- 1 procesor general	
- 1 core pt. calcule intense	- mai multe
- DDR, ierarhie cache	- DDR, ierarhie cache
- shared memory	
- multe registre	multe - registre
- SIMD	- SIMD

- ierarhie de memorii : bottleneck sist. de calcul
  - mem. gap = dif. frecv. proc. și a memoriei principale >>


CPU
registre
cache
mem. bus
mem.
i/o bus
i/o devices
viteză mare
mică
cap. mică
mare

ASC 25.IV.2024

Cache

- set-asociative

adresă furnizată în program

TAG	IDX	offset

+ bit validitate

nr. biți = $\log_2$ nr-seturi

⊛ tag adresă = tag din cache L_2

→ flush (nu așteptăm după miss)

- scrieri
  - write-through (date consistente cache-mem.)
  - write-back (doar în cache)
    - (scrieri și citiri mai rapide) L_1
    - penalizare mare la miss (x2)
      - date modif. în cache dar încă neactual. în mem.

⊛ scrieri forțate când vrem

- protocoale de coerență
  - snooping
    - la write-back → broadcast
    - înainte să fie nevoie de date se întreabă celelalte nivele dacă le au (să nu se mai ajungă la mem. princ.)
- eventual consistency

Analiza performanțelor:

- profiling: perf (linux)
- tracing: investigare bug-uri de design → load imbalance
- hotspot: fct. ce merită a fi optimizate (durează foarte mult)

benchmark → performanță HW → load imbalance

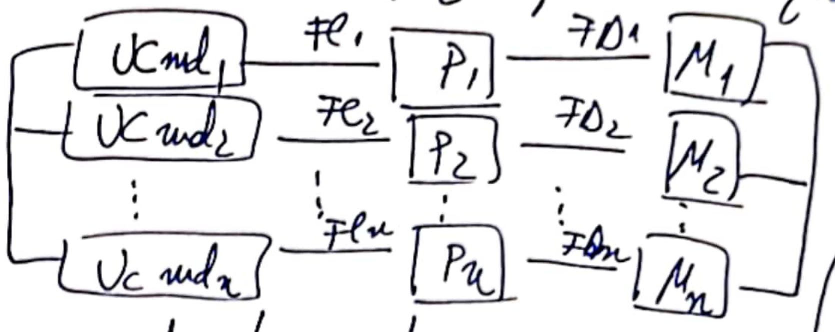
ASC

9.V.2024

MIMD — sist. — multiprocesor
— multicalculator

- shared memory
- distributed memory

cluster — sist. slab cuplat - apl. descentralizată



schimb de mesaje

MIMD = n FI & n FD

- toți proc. participă la același program

- structură eterogenă
- proc. autonome ca sist. universal (previne degradarea performanțelor)
  ↓ preluare task-uri de la alți proc.

- protecție memorie — integritate date
  (cap. adresare proc < cap. mem.)

- modularitate:
  - expandabilitate (nr. proc. se modif. fără reconfig. sist.)
  - reconfigurabilitate
    - structurală ①
    - arhitecturală ②

①: refacere stare sistem prin distrib. structurală a funcțiilor (preluare task-uri de la alți proc.)

②: schimbare caract. de bază (dim. reg., lung. cuvânt, structură UAL)

- decompoziția algoritmilor
  - manual (concepție)
  - automat (limbaje de programare concurente)

Te iubesc sunshine bubu

- planif. exec. task-uri + mapare pe structura de procesare (sincronizare!)

- siguranța în funcționare
  - disponibilitate (cap. de a răsp. cererilor)
  - integritate (consist. stării dacă apar defecte)

1 UC   n UC

(! avantaje + dezavantaje)

- flexibil
- propria mem.

asemănări - deosebiri

SIMD și MIMD — independ.

ASC 21·V·2024

- single-system image: aplicații ce rulează pe mai multe noduri dar dă impresia unui tot unitar → mapare adrese, sincronizare, comunicare

K_{map}: gestionează magistrala map bus și cererile pentru Cm-uri → nr. limitat per magistrală

cluster-ul are mai multe K_{map}-uri și map bus-uri

fiabile → asigurarea paralelismului
→ conectare matricială de clustere

- componente K_{MAP}: K_{BUS}, P_{MAP}, Link (gestionare + supraveghere mesaje)

K_{BUS} → comandă toate tranzacțiile

P_{MAP} → proc. de mapare ce face toate translatările de adresă

ASC 30.V.2024

rețea comutație → de tip poartă → crossbar (cel mai bun) ← ca timp, grad paralelism; cel mai simplu

procesor-mem: generalised connection network
- (mai multe input-uri și output-uri)
- ierarhic
- graf bipartit de conexiuni
- orice tip de conexiuni

comutatoare
- simplex (1 sg. direcție)
- duplex (în ambele direcții)

crossbar: "n" A, "m" B } ca tipuri de ~~resurse~~

- ierarhic ⇒ n × m switch-uri poartă
  - min(n, m) ← grad paralelism
  - complex: m×n ← câte sw sunt
  - ↗el ⇒ ↗preț; nu scalează debit
- neierarhic (leagă procesoare) SIMD ← comunicare între ele
  - C_n^2
  - complex: $\frac{n(n-1)}{2}$
  - grad paralelism: $\frac{n}{2}$

UNITATE DE COMANDĂ

simplif. crossbar ⇒ trunchi-k

m → linii ai} complex: k(m+n)

k → coloane — k = grad paralelism

ce nu se includ (∀ conex între A și B)

unit. de comandă: care dintre com. are loc pe fiecare k

(! nu scalează

rețeaua delta: interconectare pe nivele: A^n resurse cu B^n

- permutări elementare: cum se leagă eficient resursele
- mai scalabilă decât crossbar

$m = 2^n$ perm.

$$\underbrace{\surd_{2^n} \circ \surd_{2^n} \circ \dots \circ \surd_{2^n}}_{\text{de „n" ori}} \equiv \underbrace{(0, 1, 2, \dots, 2^n - 1)}$$

perm. identică

→ indicii elem. ce se conectează

$\surd_{2^n}(i) = (i+1) \bmod m$

$(i-1) \bmod m$

perm. circulare

shuffle / reversed shuffle / butterfly → clusterizare resurse (mai multe grupuri) → eficiență per grup

$a_0 a_1 \dots a_{m-2} a_m$ biți

→ $a_m a_0 a_1 \dots a_{m-3} a_{m-2}$

→ $a_1 a_2 \dots a_{m-2} a_{m-1} a_0$

perm. indici biți

- rețea serială:
  - scalabilă
  - cea mai ieftină
  - cel mai mare timp de propagare semnal

→ nr. intrări și nr. ieșiri să aibă aceeași bază de repr. a nr. cu exponent

→ nr. mo e mult mai mic decât ca în cazul crossbar

→ dar nu e la fel de rapid

!!! atenție la interconectarea nivelelor

→ crossbar mici în componență

cell : caract : eterogen, 2 părți de program

G.P. CPU (general purpose) — proc. dedicat

SIMD

multe registre pe CPU, mem. rapidă (doar SRAM)

asemănări / deosebiri cu GPU

benchmarking : măsurare perf. HW
comparare processor-mem sw.
rețelistică (sist. interconectare)
pt. MIMD → clustere

- HPL,
- HPCC (mai multe met. de benchmarking)
- kernel : Ax = B
- produs de matrice
- eval = apl. în sine
- sintetic : nu creez toată aplicația, testez o parte cu altceva echivalent pt calcul timp exec.

- linpack
- profiling → pt. SW

- corectitudine programe paralele → sincronizări
- GPU, SIMD, scheduler (fără L_3, nr reg., branch predictor etc.)
  - → de ce nu e eficient pt. if-uri
    elem. de sincron. după fiec. instr. → GPU, cell
    (nu se poate sinc. aceeaș unit. de comandă)
  - → complex. program

$O(n^2) \longrightarrow O(n)$ → și HW

în paralel suma de matrice → ad. la acum. proprii

se așteaptă citirea în mem:
stocare pe coloane? (nu se poate la fel pe linii)

timp / $O(n^2)$ ← proporțional cu

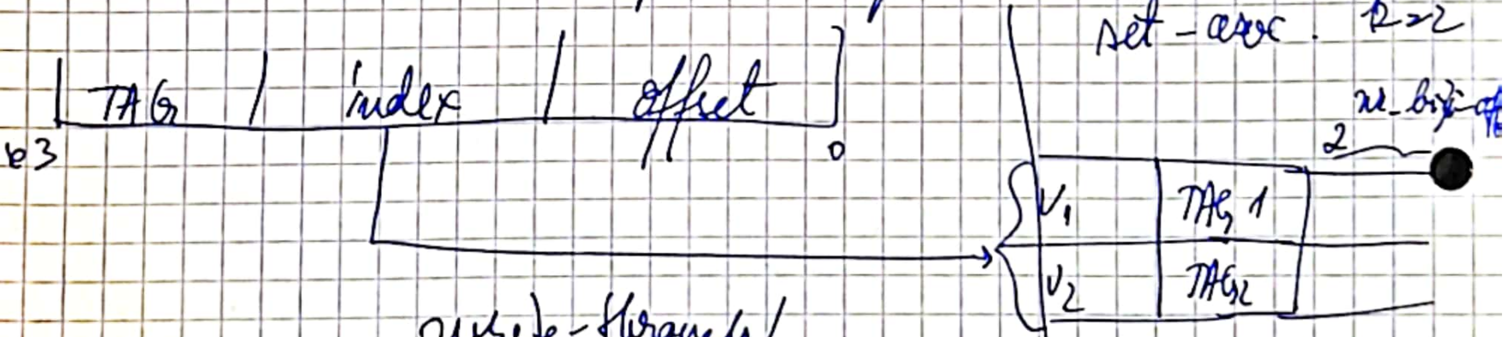
=> pt. suma pe linii se transpune matricea

sau: org. date conform permutării circulare
(pt. ambele cazuri)

! unit. de comandă crossbar-uri
să facă leg. corecte

┌ localizare date (se schimbă la fiecare pas)

- taxonomia Flynn : % din program paralelizat
speed-up obținut proporțional cu
- tendința de multicore (MIMD) - de ce
  - construcție ↓ proiectării
  - dif. față de SIMD
SISD în ce situații sunt avantajoase fiec. dintre ele
- MISD??
- SIMD mai ieftin decât MIMD
- cum funcționează cache-ul
(+ loc. spațială și temporală)



write-through/
write back

prob. coerență

bit de dirty